

G17 BEE

Frameworks in Node.js

Express Introduction

Serving Static files in express

Updated with Video recordings

Frameworks of Node.js

Framework	Type	Key Features	Best For	Official Website
Express.js	Minimalist, Flexible	Simple routing, Middleware support, Extensible, Template engine support	REST APIs, SPAs, Web apps	expressjs.com
NestJS	Full-featured, Modular	TypeScript support, Modular architecture, Microservices, Dependency injection	Enterprise apps, Microservices, APIs	nestjs.com
Koa.js	Lightweight, Modern	Async/await support, Middleware pipeline, Flexible, Lightweight	Scalable apps, APIs with fine-grained control	koajs.com
Sails.js	MVC, Data-driven	MVC architecture, WebSocket support, RESTful routing, ORM (Waterline)	Data-driven apps, Real-time apps, APIs	sailsjs.com
Hapi.js	Powerful, Configurable	Input validation, Caching, Authentication, Plugin-based	APIs, Enterprise-level apps, Highly configurable apps	hapi.dev
Next.js	Full-stack, SSR, React	Server-side rendering (SSR), Static site generation (SSG), API routes, Routing	React apps, SSR, Full-stack, JAMstack apps	nextjs.org

Express.js

- **Type:** Minimalist and flexible
- **Overview:** Express.js is one of the most widely used frameworks in the Node.js ecosystem. It is minimal, unopinionated, and provides a simple API to build robust web applications and APIs. Express handles routing, middleware, template engines, and much more.
- **Use Case:** Ideal for building RESTful APIs, single-page applications (SPAs), and server-side applications.

Key Features:

- Simple routing
- Middleware support
- Template engine support (e.g., EJS, Pug)
- Robust routing system
- Extensible with many third-party packages

Response Methods in Express

Method	Purpose	Common Use Cases
<code>res.send()</code>	Sends a generic response (string, object, or buffer).	Sending text, JSON, or binary data.
<code>res.json()</code>	Sends a JSON response.	Returning JSON data in APIs.
<code>res.jsonp()</code>	Sends a JSONP response.	Supporting cross-origin requests.
<code>res.sendFile()</code>	Sends a file as the response.	Serving static files like HTML, images, PDFs, etc.
<code>res.status()</code>	Sets the HTTP status code.	Indicating the success or failure of the request.
<code>res.redirect()</code>	Sends an HTTP redirect response.	Redirecting to another URL.
<code>res.render()</code>	Renders a view and sends the HTML output.	Rendering dynamic HTML views using a templating engine.
<code>res.set()</code>	Sets HTTP headers.	Customizing response headers (e.g., for caching or content type).
<code>res.type()</code>	Sets the Content-Type header.	Setting content type shorthand (e.g., JSON, HTML).
<code>res.sendStatus()</code>	Sends a status code with a default message.	Sending just a status code (e.g., 404 Not Found).
<code>res.location()</code>	Sets the Location header (often with 3xx/201 responses).	Indicating the URI of a new or redirected resource.
<code>res.vary()</code>	Sets the Vary header.	Indicating response variations based on request headers.

Example

```
//initialyse project
//>npm init -y
//download express module
//>yarn add express --save
//import express module
let express = require('express')
//create rest object
let app = express() //where app is rest object
//create get request
app.get("/",(req,res)=>{
  console.log('Default get message')
  res.json({'message':'default get request'})
})
```

```

//create one more get request
app.get("/fetch", (req, res) => {
  res.send({ 'message': 'fetch get request' })
})
//create post request
app.post("/", (req, res) => {
  res.send({ 'message': 'default post request' })
})
//create one more post request
app.post("/login", (req, res) => {
  res.send({ 'message': 'login post request' })
})

//create a port
let port = 8080
//assign port no
app.listen(port, () => {
  console.log('Server listening port no ', port)
})

/*
  Test urls with postman
  http://localhost:8080           Default GET
  http://localhost:8080/fetch    fetch GET
  http://localhost:8080         Default POST
  http://localhost:8080/login    login POST
*/
07_01 Express Server      https://shorturl.at/ZbLRx

```

=====

Serve Static Files

=====

- static() is the predefined function from express.
- This function is used to set the target directory.
- Argument to this function is path.

Directory Structure

```

<>
  public
    - notes.txt
    - index.html
  - server.js

```

note.txt

these are the important notes of Node.js

index.html

```

<!DOCTYPE html>
<html>
  <head>
    <title>Express static</title>
    <link rel="icon" href="https://expressjs.com/images/favicon.png"
type="image/x-icon">
  </head>
  <body>

```

```
    <h1 style="color: red;">Welcome to static files in express</h1>
  </body>
</html>
```

```
***server.js***
// Import the express module
const express = require('express')
// Create an Express application
const app = express()
// Serve static files from the 'public' directory
app.use(express.static('public'))
// Define a route to handle a basic request to the root of the app
app.get("/", (req, res) => {
  res.send("Welcome to the static files server!")
})
// Start the server and listen on port 8080
const port = 8080
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`)
})
07_02 Serving Static files express    https://shorturl.at/kdEb1
```